



UNIVERSIDAD DE LAS FUERZAS ARMADAS - ESPE

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

Ingeniería de la Seguridad del Software

Integrantes: Mateo Sosa

NRC: 30808

Fecha: June 12, 2026

Contents

Objetivo	4
Topología de Prueba	4
Objetivos de Aprendizaje	4
Marco Teórico	4
AES	4
Criptología	4
Criptografía	5
Criptoanálisis	5
Concepto de Criptografía Simétrica	5
Topología de prueba	5
Desarrollo	6
Preparación del entorno	6
Crear un archivo de texto para cifrar	6
Cifrado con AES	6
Verificación del cifrado	7
Descifrado del archivo	8
Reflexión y análisis	8
Actividades complementarias	8
Análisis y ejecución del algoritmo básico AES-256 en Python	8
Actividad de modificación y mejoramiento del algoritmo propuesto	10
Pregunta	10
Investigación	11

List of Figures

1	Verificación de Versión de <code>openssl</code>	6
---	---	---

2	Creación de Archivo de Texto a Cifrar	6
3	Encriptado de Archivo con AES	7
4	Verificación de Archivo Cifrado	7
5	Contenido de Mensaje Cifrado	7
6	Desencriptado de Mensaje Cifrado con Contraseña	8
7	Contenido de Mensaje Descifrado	8
8	Instalación de Librería cryptography	9
9	Ejecución de Script Python del Ejercicio 6	9
10	Ejecución de Script Python de Ejercicio 7	10
11	Creación del Archivo del Ejercicio 8	11
12	Encriptado del Archivo mediante Diversos Algoritmos	12
13	Revisión de Archivos Encriptados mediante hexdump	12

List of Tables

Objetivo

Aplicar el cifrado simétrico utilizando el algoritmo **AES (Advanced Encryption Standard)** en un entorno virtual controlado con Kali Linux, comprendiendo su funcionamiento y seguridad.

Topología de Prueba

Entorno Virtual de Red: Kali Linux y OpenSSL

Objetivos de Aprendizaje

1. Comprender el funcionamiento del algoritmo de cifrado simétrico AES.
2. Analizar e implementar una versión básica de AES en Python.
3. Probar el algoritmo con claves de 128, 192 y 256 bits.
4. Aplicar AES en un contexto de red para proteger la información.
5. Identificar posibles mejoras al código de cifrado implementado en Python.
6. Simular ataques para evaluar la efectividad de las técnicas de cifrado.

Marco Teórico

AES

El AES (Advanced Encryption Standard) es un algoritmo de cifrado simétrico adoptado como estándar por el NIST en 2001. Sustituyó al DES debido a sus debilidades frente a ataques por fuerza bruta. AES está basado en el algoritmo Rijndael, propuesto por Joan Daemen y Vincent Rijmen, y emplea claves de 128, 192 o 256 bits con bloques fijos de 128 bits. Dependiendo del tamaño de la clave, realiza 10, 12 o 14 rondas de operaciones criptográficas (Daemen & Rijmen, 2002; Stallings, 2017).

Las operaciones internas incluyen sustituciones, permutaciones y combinaciones lineales a nivel de byte: SubBytes, ShiftRows, MixColumns y AddRoundKey. En redes informáticas, AES se utiliza para cifrar comunicaciones seguras entre hosts o dispositivos, siendo clave en protocolos como TLS, IPsec y VPNs.

Criptología

La criptología es la ciencia que estudia los métodos y técnicas utilizadas para proteger la información mediante el uso de códigos y cifrados, con el fin de garantizar la triada CID y el No repudio de los datos. Abarca dos disciplinas principales: la criptografía (cifrado de la información) y el criptoanálisis (estudio de la decodificación de mensajes cifrados sin conocer la clave).

Criptografía

La criptografía es una rama de la criptología que se enfoca en el diseño y estudio de algoritmos matemáticos y protocolos que permiten transformar la información en un formato seguro y protegido contra accesos no autorizados.

Criptoanálisis

El criptoanálisis es la parte de la criptología que se dedica al estudio de sistemas criptográficos con el fin de encontrar debilidades en dichos sistemas y romper su seguridad sin el conocimiento de información secreta.

Concepto de Criptografía Simétrica

La criptografía simétrica es un tipo de criptografía en la que se utiliza la misma clave para cifrar y descifrar la información. Esto significa que ambas partes (el remitente y el receptor) deben compartir una clave secreta previamente acordada. El principal desafío de la criptografía simétrica es la gestión de claves, ya que la clave debe ser transmitida de forma segura sin ser interceptada.

OpenSSL es una herramienta de software de código abierto ampliamente utilizada en sistemas Linux (y otros sistemas operativos) para la implementación de funciones criptográficas. Ofrece una biblioteca criptográfica robusta que proporciona un conjunto de algoritmos de cifrado, funciones de seguridad y utilidades para la gestión de certificados **SSL/TLS**, la generación de claves y la creación de firmas digitales. Entre sus principales usos se tiene:

- OpenSSL permite realizar cifrado y descifrado de datos utilizando una variedad de algoritmos criptográficos, como **AES**, **DES**, **RSA**, **Blowfish**, entre otros.
- OpenSSL permite generar claves públicas y privadas para criptografía simétrica y asimétrica.
- OpenSSL permite generar certificados digitales utilizados en protocolos como **SSL/TLS** para asegurar la comunicación en redes.

Topología de prueba

- **VM Cliente (Ubuntu Desktop):** Ejecutará un programa Python para cifrar y enviar un mensaje usando AES.
- **VM Servidor (Ubuntu Server):** Recibirá y descifrá el mensaje enviado por el cliente, también usando Python.
- **VM Atacante (Kali Linux):** Capturará el tráfico de red (usando Wireshark o tcpdump) e intentará analizar su contenido.
- **Python 3 instalado en cliente y servidor.**
- **Biblioteca PyCryptodome:**

Desarrollo

Preparación del entorno

1. Inicie su máquina virtual Kali Linux y abra una terminal.
2. Verifique que OpenSSL esté instalado en tu máquina. Para esto, escriba el siguiente comando en la terminal:

```
openssl version
```

Si OpenSSL no está instalado, instálalo ejecutando:

```
sudo apt-get update
```

```
sudo apt-get install openssl
```

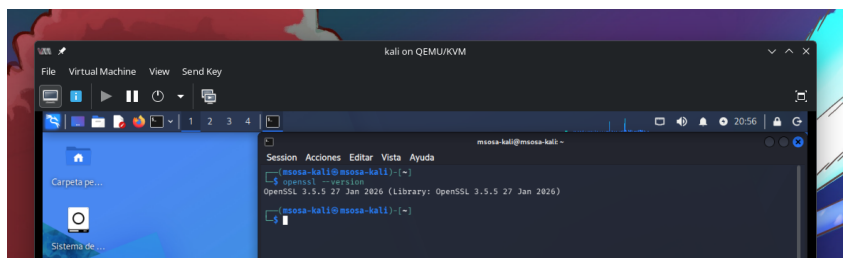


Figure 1: Verificación de Versión de openssl

Crear un archivo de texto para cifrar

1. Cree un archivo de texto con el siguiente contenido:

```
echo "Este es un mensaje secreto que vamos a cifrar usando AES" > mensaje.txt
```

2. Verifique que el archivo se haya creado correctamente:

```
cat mensaje.txt
```

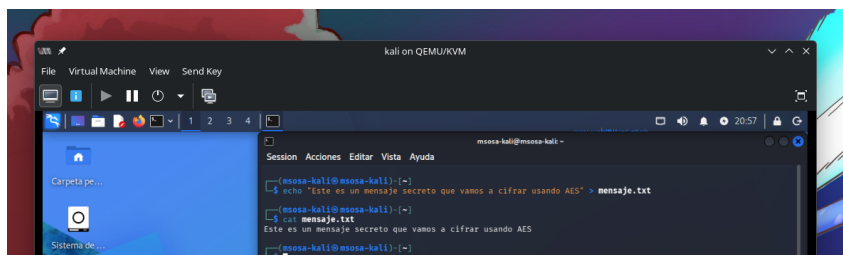


Figure 2: Creación de Archivo de Texto a Cifrar

Cifrado con AES

1. Para cifrar el archivo mensaje.txt usando **AES-256-CBC** (AES con un tamaño de clave de 256 bits y el modo de operación CBC), utilice el siguiente comando:

```
openssl enc -aes-256-cbc -salt -in mensaje.txt -out mensaje_cifrado.txt
```


Descifrado del archivo

1. Para descifrar el archivo, use el siguiente comando:

```
openssl enc -d -aes-256-cbc -in mensaje_cifrado.txt -out mensaje_descifrado.txt
```

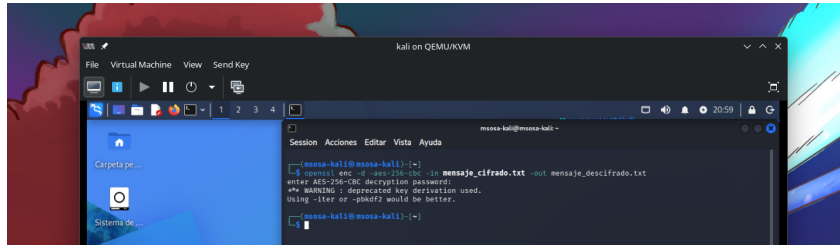


Figure 6: Desencriptado de Mensaje Cifrado con Contraseña

Se te pedirá que ingreses la misma contraseña utilizada durante el cifrado.

2. Una vez completado el proceso, Verifique que el archivo haya sido descifrado correctamente:

```
cat mensaje_descifrado.txt
```

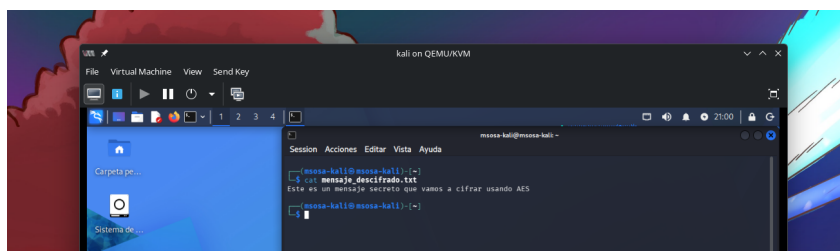


Figure 7: Contenido de Mensaje Descifrado

Reflexión y análisis

1. **Seguridad:** La criptografía simétrica es eficiente en términos de velocidad, pero su principal vulnerabilidad es la **gestión de claves**. Ambas partes que cifran y descifran deben compartir la misma clave, lo que puede generar problemas de seguridad si la clave es interceptada.

Actividades complementarias

Análisis y ejecución del algoritmo básico AES-256 en Python

Objetivo: Comprender el funcionamiento básico del cifrado simétrico AES-256 en modo GCM, mediante la ejecución de un programa en Python que permite cifrar y descifrar información.

Instalar la librería

```
pip install cryptography
```



```
(venv) msosa@fedora:~/Documents/ESPE/seguridad-30808/P2/Laboratorios/3$ pip install cryptography
Collecting cryptography
  Downloading cryptography-49.0.0-cp311-abi3-manylinux_2_34_x86_64.whl.metadata (4.3 kB)
Collecting cffi>=2.0.0 (from cryptography)
  Downloading cffi-2.0.0-cp314-cp314-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (2.6 kB)
Collecting pycparser (from cffi>=2.0.0->cryptography)
  Downloading pycparser-3.0-py3-none-any.whl.metadata (8.2 kB)
Downloading cryptography-49.0.0-cp311-abi3-manylinux_2_34_x86_64.whl (4.7 MB)
3.4/4.7 MB 1.1 MB/s eta 0:00:02
```

Figure 8: Instalación de Librería cryptography

```
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
import os

# Generación de clave AES-256 (32 bytes = 256 bits)
key = AESGCM.generate_key(bit_length=256)
aesgcm = AESGCM(key)

# Nonce (vector de inicialización)
nonce = os.urandom(12)

# Mensaje original
mensaje = "Seguridad informática con AES-256"
datos = mensaje.encode("utf-8")

# CIFRADO
ciphertext = aesgcm.encrypt(nonce, datos, None)

print("MENSAJE ORIGINAL:", mensaje)
print("CLAVE:", key.hex())
print("NONCE:", nonce.hex())
print("CIFRADO:", ciphertext.hex())

# DESCIFRADO
texto_descifrado = aesgcm.decrypt(nonce, ciphertext, None)
print("DESCIFRADO:", texto_descifrado.decode("utf-8"))
```

```
(venv) msosa@fedora:~/Documents/ESPE/seguridad-30808/P2/Laboratorios/3$ python ej6.py
MENSAJE ORIGINAL: Seguridad informática con AES-256
CLAVE: bbf52f89725b23f3b1e4efab0b1b067cfa68ca5d173efb4bbccd9d58b52b57bf
NONCE: 6f70895a797b44f726ce65e9
CIFRADO: 08e70a9565e1c7acd43be8a625603ef235213d7a83849a9a217edc51d00955b1f16dd44ca6118417544267fb700672ec9d37
DESCIFRADO: Seguridad informática con AES-256
```

Figure 9: Ejecución de Script Python del Ejercicio 6

Responder:

- ¿Por qué la clave AES-256 tiene exactamente 32 bytes?

Porque AES-256 utiliza una clave de 256 bits y cada byte equivale a 8 bits, por lo que $256 \div 8 = 32$ bytes.

- ¿Qué función cumple el nonce en el proceso de cifrado?

El nonce introduce aleatoriedad en el cifrado para que un mismo mensaje cifrado varias veces produzca resultados diferentes.

- ¿Qué ocurre si se utiliza la misma clave y nonce en dos mensajes diferentes?

Se compromete la seguridad del cifrado, ya que un atacante podría obtener información sobre los mensajes y facilitar ataques criptográficos.

- ¿Por qué AES se considera un algoritmo simétrico?

Porque utiliza la misma clave tanto para cifrar como para descifrar la información.

- ¿Qué ventaja tiene el modo GCM frente a CBC?

GCM proporciona confidencialidad e integridad de los datos, mientras que CBC solo proporciona confidencialidad y requiere mecanismos adicionales para verificar la integridad.

Actividad de modificación y mejoramiento del algoritmo propuesto

Objetivo: Modificar el programa base para analizar la robustez del cifrado AES-256-GCM y comprender el impacto de cambios en sus componentes.

Actividades propuestas

Actividad 1: Manipulación del ciphertext

Modificar el código agregando:

```
ciphertext = bytearray(ciphertext)
ciphertext[0] ^= 1
ciphertext = bytes(ciphertext)
```

```
(env) msosa@fedora: ~/Documents/ESPE/seguridad-30808/P2/Laboratorios/$ python ej7.py
MENSAJE ORIGINAL: Seguridad informática con AES-256
CLAVE: 5f271d74f9e1ee2245a724d0e0fb5c72685f3bbe4726b445idf7b82d811c36
NONCE: 42870812434546fb6cf349
CIPHERDATA: 983137467c436694b4d813280a30170603eb47f1b62fc1b1984e623989b6441424ca359f89a8860c364fd6f3a0b54f461c0
Traceback (most recent call last):
  File "/home/msosa/Documents/ESPE/seguridad-30808/P2/Laboratorios/3/ej7.py", line 28, in <module>
    texto_descifrado = aesgcm.decrypt(nonce, ciphertext, None)
  File "/usr/lib/python3.10/site-packages/cryptography/hazmat/primitives/aes_gcm.py", line 100, in decrypt
    raise InvalidTag
cryptography.exceptions.InvalidTag
```

Figure 10: Ejecución de Script Python de Ejercicio 7

Pregunta

¿Qué ocurre al intentar descifrar el mensaje modificado?

Genera un error proveniente de la librería instalada al no conocer cómo poder descifrar el mensaje.

Investigación

Investigue los modos de operación ECB (Electronic Codebook), CBC (Cipher Block Chaining), CFB (Cipher Feedback), OFB (Output Feedback) **GCM (Galois/Counter Mode)** en AES; cifre un archivo de texto utilizando al menos dos de ellos con OpenSSL en Kali Linux; compare los resultados visualmente con hexdump; y redacte una breve reflexión sobre las diferencias observadas y su impacto en la seguridad de la información.

Modo	Descripción	Ventajas	Desventajas	Uso recomendado
ECB (Electronic Codebook)	Cifra cada bloque de forma independiente.	Simple y rápido.	Bloques iguales producen cifrados iguales, revelando patrones.	No recomendado para datos sensibles.
CBC (Cipher Block Chaining)	Cada bloque depende del bloque anterior mediante un IV.	Oculto patrones y mejora la seguridad respecto a ECB.	Requiere IV y no permite paralelismo completo en el cifrado.	Archivos y almacenamiento general.
CFB (Cipher Feedback)	Convierte un cifrador de bloques en uno de flujo mediante retroalimentación.	Permite cifrar datos de tamaño variable.	Un error puede propagarse temporalmente.	Comunicaciones en tiempo real.
OFB (Output Feedback)	Genera un flujo de claves independiente del texto plano.	Los errores no se propagan.	Reutilizar IV compromete la seguridad.	Canales con ruido o transmisión de datos.
GCM (Galois/Counter Mode)	Combina cifrado en modo contador con autenticación.	Confidencialidad, integridad y alto rendimiento.	Requiere manejo correcto de nonces.	Aplicaciones modernas, TLS, VPN y almacenamiento seguro.

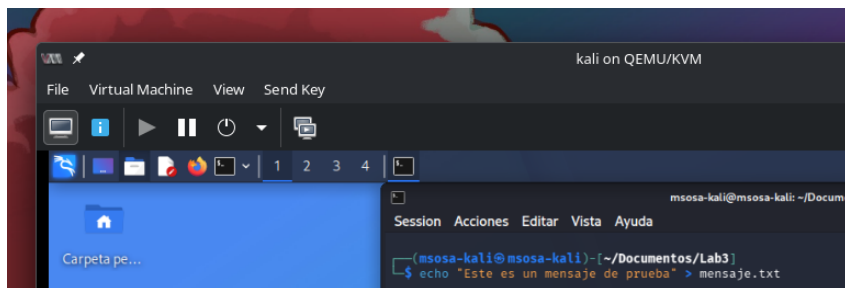


Figure 11: Creación del Archivo del Ejercicio 8

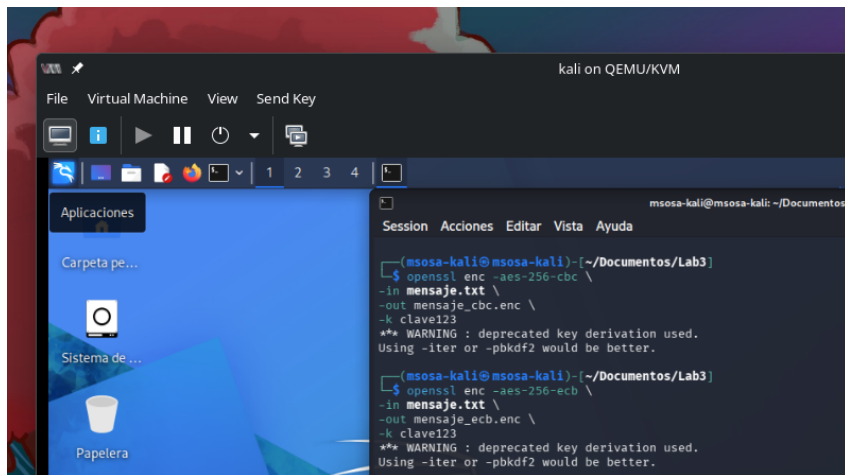


Figure 12: Encriptado del Archivo mediante Diversos Algoritmos

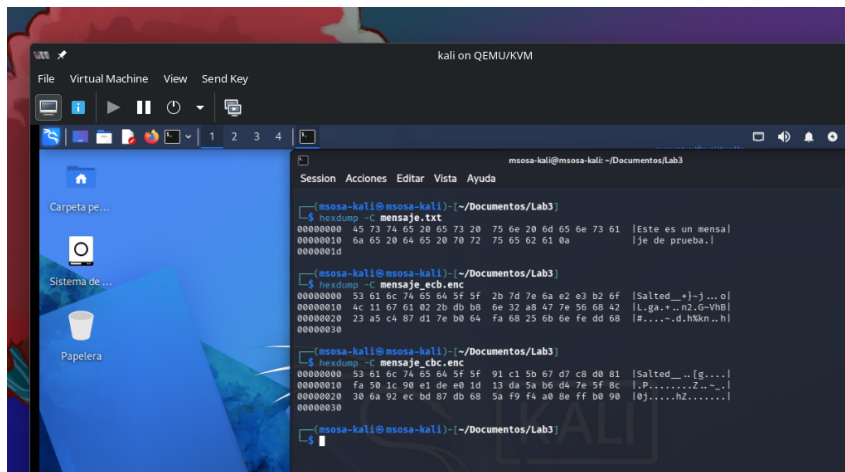


Figure 13: Revisión de Archivos Encriptados mediante hexdump

Al comparar los archivos cifrados mediante `hexdump`, se observa que todos los modos generan datos aparentemente aleatorios, pero ECB puede revelar patrones cuando existen bloques repetidos, mientras que CBC y GCM producen resultados más seguros al incorporar información adicional como IV o nonce. GCM destaca porque además de cifrar los datos garantiza su integridad, evitando modificaciones no autorizadas, por lo que es uno de los modos más utilizados actualmente en aplicaciones y protocolos modernos.

Conclusiones:

1. Esta práctica ha proporcionado una introducción a la criptografía simétrica utilizando el algoritmo AES. A través de la utilización de **OpenSSL**, los estudiantes pudieron cifrar y descifrar un mensaje, aprendiendo los conceptos clave detrás de los **métodos de cifrado** y los **modos de operación** utilizados en criptografía moderna. Además, se destacó la importancia de la **gestión de claves** en criptografía simétrica.
2. La criptografía simétrica es eficiente en términos de velocidad, pero su principal vulnerabilidad es la **gestión de claves**. Ambas partes que cifran y descifran deben compartir la misma clave, lo que puede generar problemas de seguridad si la clave es interceptada.

3. AES es un algoritmo seguro, pero su seguridad depende de su correcta implementación.
4. El manejo incorrecto de claves puede comprometer completamente el sistema.

Referencias Bibliográficas:

- Stallings, W. (2017). *Cryptography and Network Security: Principles and Practice* (7th ed.). Pearson.
- Ferguson, N., Schneier, B., & Kohno, T. (2010). *Cryptography Engineering: Design Principles and Practical Applications*. Wiley.
- OpenSSL Project. (n.d.). *OpenSSL Documentation*. Retrieved from <https://www.openssl.org/docs/>
- Schneier, B. (1996). *Applied Cryptography: Protocols, Algorithms, and Source Code in C* (2nd ed.). Wiley.